

Lecture 17 - Fusion, Layout Propagation

- AI/NPU Compiler 를 위한 MLIR
- Transformer MLP fusion 후보 분석
- 산출물 : fusion/layout/cost contract + pseudo IR

오늘의 위치

- 14 강 Vector/Tensorization: native tile 로 mapping
- 15 강 Transform Dialect: schedule IR
- 16 강 Quantization: numeric contract
- 17 강 : fusion/layout decision 으로 NPU performance 를 결정

Learning Objectives

- Fusion 을 legality + cost + layout contract 로 정의
- Transformer MLP 에서 fusion 후보를 체계적으로 분류
- Layout propagation 과 materialization boundary 설계
- SRAM/ACCUM footprint 와 traffic saving 계산
- FileCheck 기반 regression test 작성

왜 Fusion 인가 ?

- Physical AI edge NPU 는 DRAM bandwidth 가 성능 병목인 경우가 많음
- Intermediate tensor write/read 제거가 latency 와 power 를 줄임
- Epilogue 를 accumulator/SRAM 내부에서 처리하면 command 수와 DMA 가 감소
- 단 , fusion 이 live range 와 SRAM pressure 를 증가시킬 수 있음

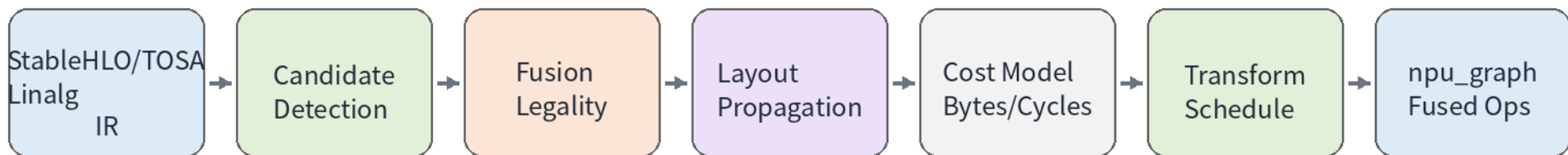
Rule: fusion must reduce traffic without violating SRAM, layout, or numeric contracts.

Fusion 의 세 수준

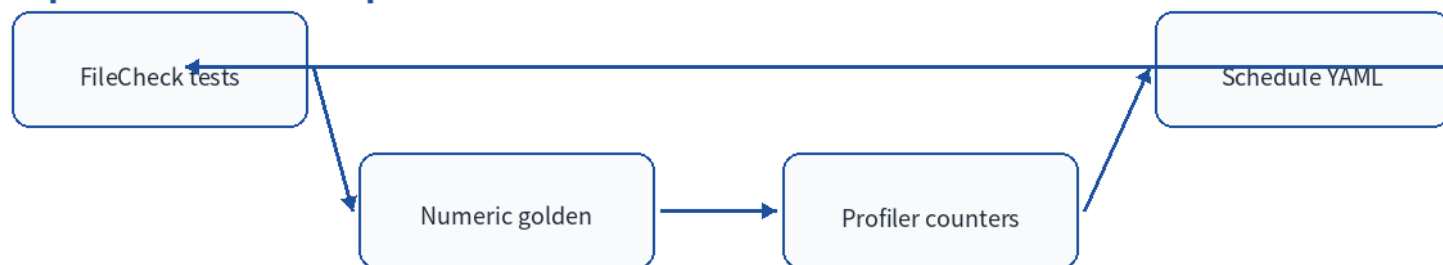
- Op fusion: IR 에서 semantic op 를 결합
- Kernel fusion: 하나의 NPU kernel 로 lowering
- Command fusion: command buffer 에서 DMA/compute/barrier 를 결합
- 각 수준마다 verifier 와 test 가 달라야 함

Fusion Pipeline Overview

Lecture 17: Fusion + Layout Propagation Pipeline



Compiler feedback loop



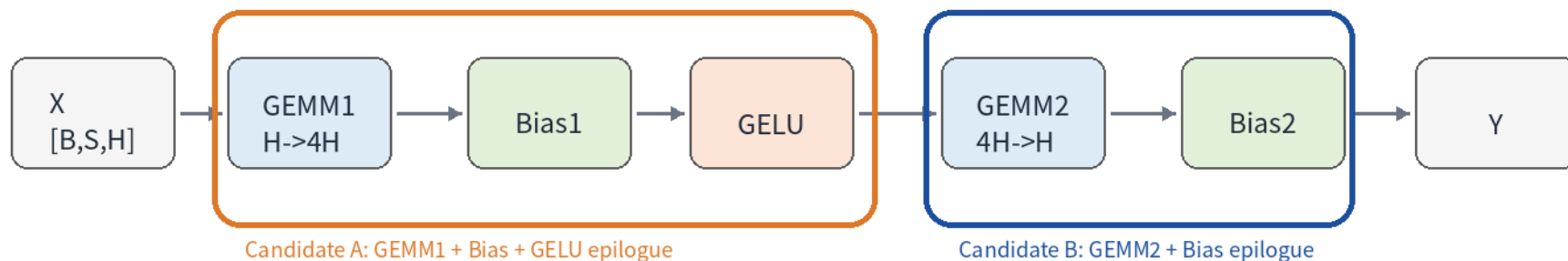
Key output: fusion groups + propagated layout contracts + materialization points + NPU schedule plan

Transformer MLP 구조

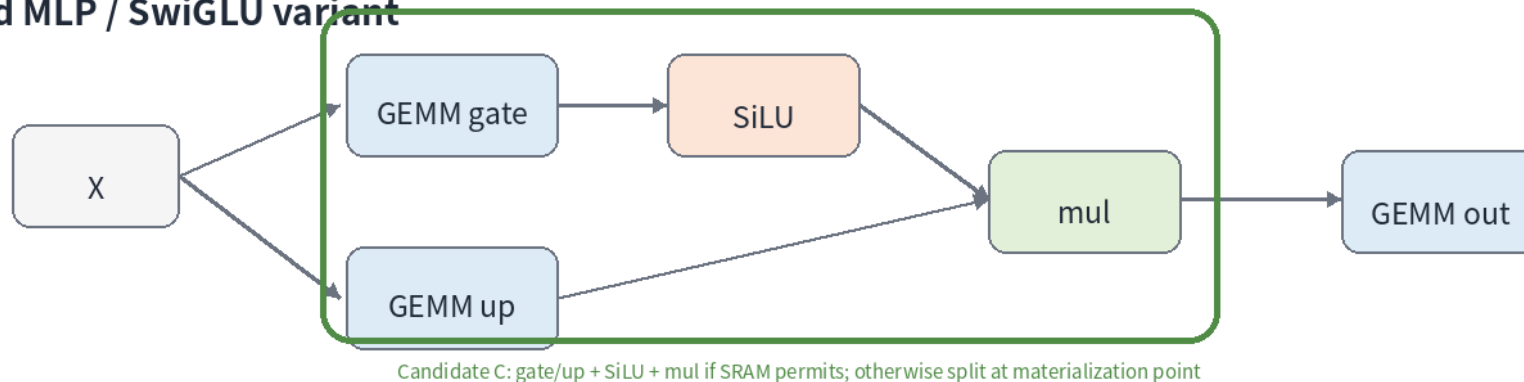
- Classic: $X \rightarrow \text{GEMM1} \rightarrow \text{Bias} \rightarrow \text{GELU} \rightarrow \text{GEMM2} \rightarrow \text{Bias}$
- Gated: gate/up projection $\rightarrow \text{SiLU} \rightarrow \text{Mul} \rightarrow \text{output projection}$
- MLP intermediate 는 H 보다 4x 이상 커서 traffic 영향이 큼

MLP Fusion Candidates

Transformer MLP Fusion Candidates



Gated MLP / SwiGLU variant



Legality != Profitability

- Legal: shape/dtype/layout/quant 조건을 만족
- Profitable: saved traffic and cycles > extra live range/repack cost
- NPU compiler 는 두 조건을 분리해 debug 해야 함

Keep legality and profitability as separate debug-visible decisions.

Fusion Legality Fields

- rank/static shape/dynamic dims
- dtype and accumulator type
- broadcast semantics
- per-channel quant axis
- layout compatibility
- single-use or profitable multi-use

Use-Def 기반 후보 탐지

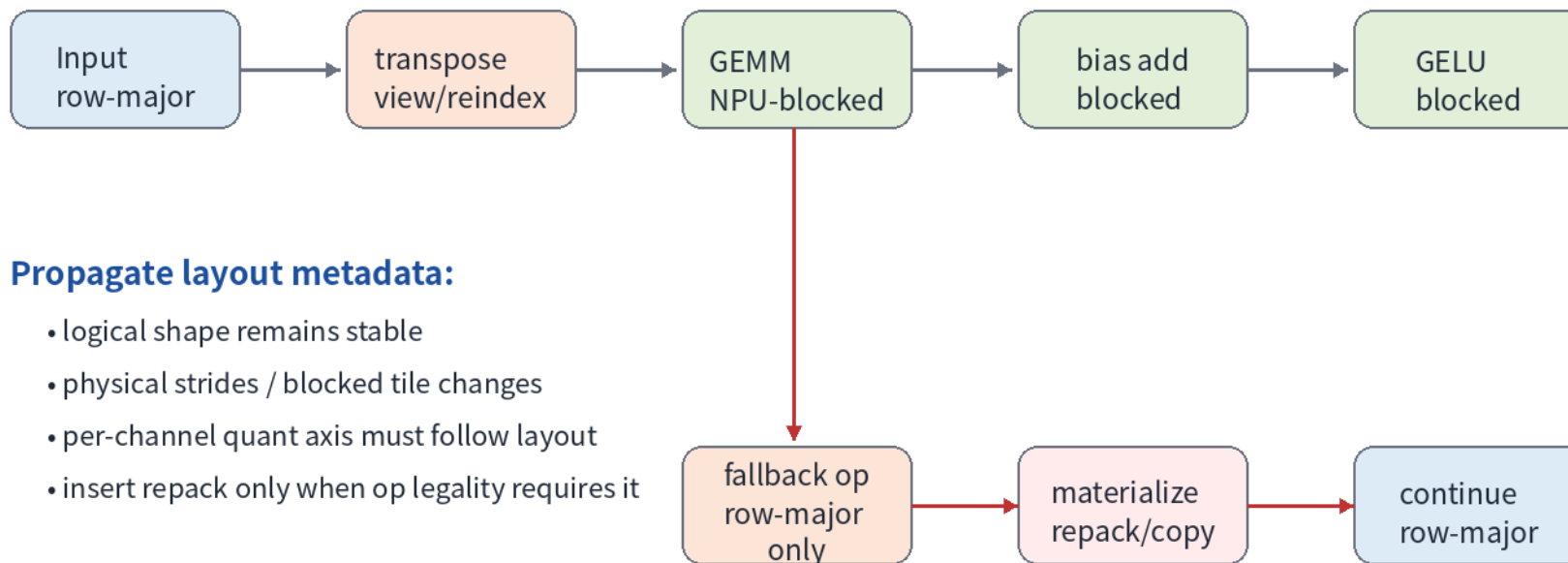
- Root op 를 activation 또는 consumer matmul 로 잡음
- Producer chain 을 거슬러 matmul/bias/elementwise 패턴 확인
- multiple consumer 는 cost model 로만 허용
- IR mutation 전에는 match-only 로 실패 조건 수집

Layout Propagation 개념

- Logical shape 와 physical layout 을 분리
- Elementwise/view-like op 는 layout-preserving 인 경우가 많음
- Consumer preferred layout 을 producer 로 역전파 가능
- 불가피한 boundary 에서만 explicit materialization 삽입

Layout Propagation Graph

Goal: push one physical layout through cheap ops and materialize only at unavoidable boundaries.



Propagate layout metadata:

- logical shape remains stable
- physical strides / blocked tile changes
- per-channel quant axis must follow layout
- insert repack only when op legality requires it

NPU compiler contract:

update fusion legality, DMA descriptors, vector maps, and runtime ABI.

Layout Propagation Rules

- Propagate through elementwise, relu/gelu/silu approximation, scalar mul
- Propagate through reshape if view-like and axis mapping known
- Stop at unsupported reduction/gather/scatter/fallback
- Insert `npu_graph.layout_materialize` with reason attr

Quantization Axis 문제

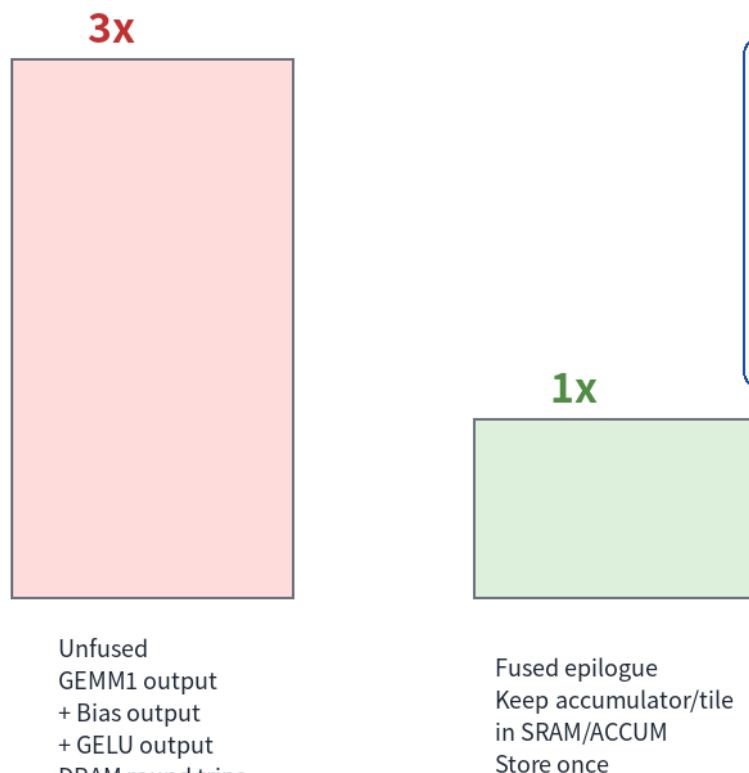
- Per-channel scale axis 가 layout 변경 후에도 정확해야 함
- N dimension blocking 은 channel axis 를 유지하는지 검증 필요
- INT8/INT4 pack layout 은 scale layout 도 함께 정의해야 함
- Mismatch 는 silent accuracy bug 가 될 수 있음

Quant axis mapping is part of layout propagation, not an afterthought.

Cost Model: Traffic

Memory Traffic Reduction by Fusion

Example: MLP intermediate H=4096, expansion=4, sequence tile=128, FP16 activations.



Traffic model

- bytes = reads + writes + repacks
- fusion gain $\approx$ removed_intermediate_writes
- but only if SRAM footprint $\leq$ budget
- and layout repack cost < saved DRAM traffic

Key idea: fusion is not always faster. A cost model must include extra live range, bank conflicts, layout materialization, and lost parallelism.

Cost Model: SRAM/ACCUM

- Input/weight/output tile + accumulator + epilogue temp + double buffer
- SwiGLU 는 gate/up branch live range 가 길어짐
- ACCUM tile precision 이 SRAM 보다 더 제한적일 수 있음
- Spill 발생 시 fusion benefit 이 크게 감소

Cost Model: Cycles

- $\text{compute cycles} = \text{MAC} / \text{effective TOPS}$
- $\text{DMA cycles} = \text{bytes} / \text{effective bandwidth}$
- bank conflict and tensor core occupancy 포함
- $\text{critical path} \hat{=} \max(\text{compute}, \text{DMA}) + \text{barrier overhead}$

Classic MLP Case Study

- GEMM1 output: [tokens, 4H] 로 큼
- Bias+GELU 는 ideal epilogue fusion 후보
- GEMM2 는 GELU output layout 을 input layout 으로 요구
- Blocked activation 을 GEMM2 까지 유지할지 cost model 로 결정

SwiGLU Case Study

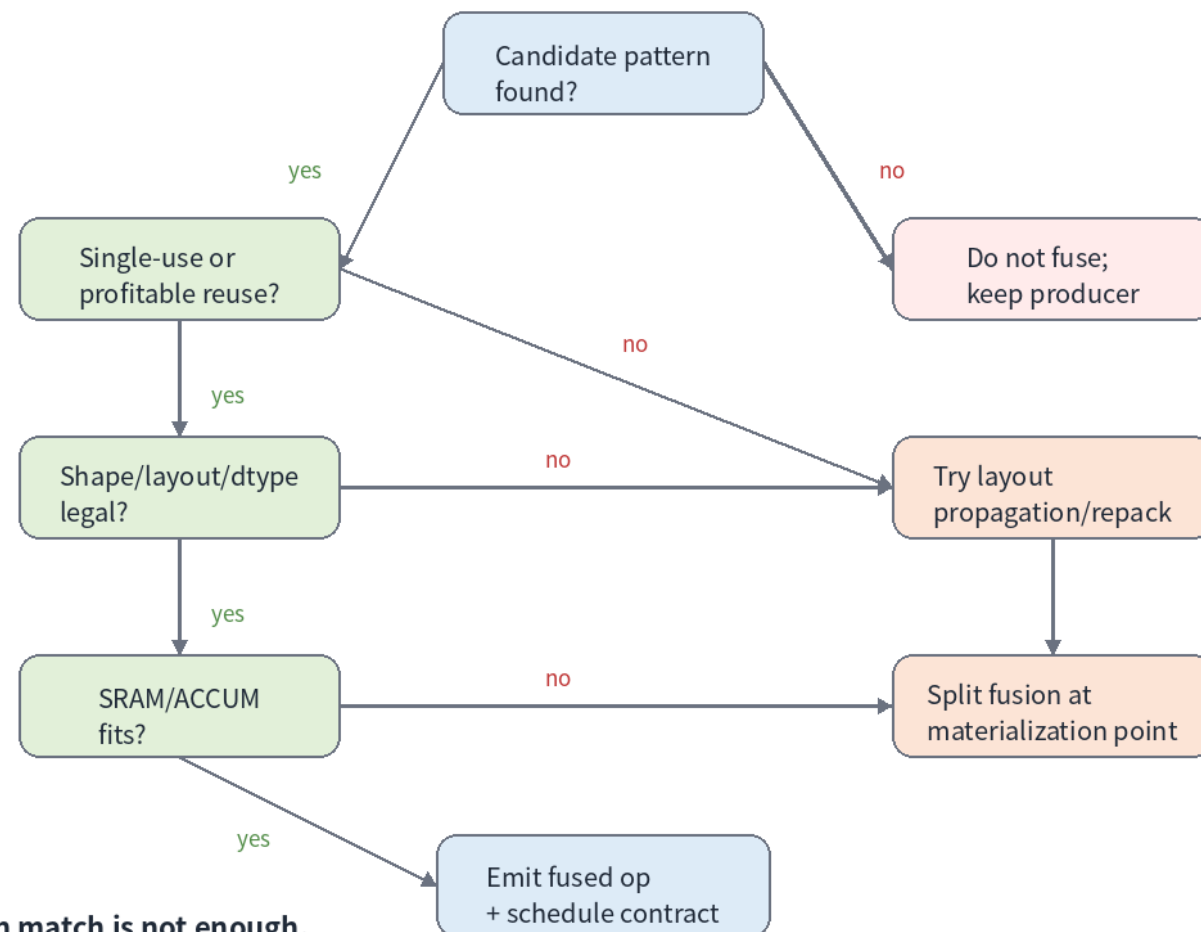
- Gate and up projections produce two large intermediates
- $\text{SiLU}(\text{gate}) * \text{up}$ 는 tile-local fusion 후보
- 두 branch 동시 live range 가 SRAM 을 압박
- branch streaming 또는 split materialization 정책 필요

Weight Prepack

- Static weight 는 compile/deploy time prepack 가능
- Dynamic weight 는 runtime cache 와 invalidation 정책 필요
- Prepack layout 은 tensorization map 과 DMA descriptor 에 반영
- Quant scale/zero-point 도 packed axis 에 맞춰 저장

Weight prepack changes runtime ABI and model packaging.

Fusion + Layout Propagation Decision Tree



Code review rule: pattern match is not enough.
Every fused op needs a legality contract and cost justification.

IR Contract: fused op

- epilogue attr: bias_gelu_tanh, bias_relu, requant_clamp
- layout attrs: lhs/rhs/out
- accumulator attr: fp32/int32
- fusion_id/debug metadata
- materialization reason attrs

Transform Dialect 와 연결

- Transform IR 로 tile/fuse/vectorize 순서를 명시
- Fusion decision 은 transform schedule parameter 가 될 수 있음
- Schedule YAML 과 transform IR 을 연결하면 autotuning 에 유리
- Payload IR 과 schedule IR 을 분리해 review 가능

Testing Strategy

- Positive: fused op 생성과 attr 확인
- Negative: unsupported shape/layout/quant 에서는 no fusion
- CHECK-NOT: unfused bias/activation 제거
- materialization op count 제한
- numeric golden test 로 approximation 검증

Tests should catch both over-fusion and missed materialization.

Debug Recipe

- 1. print candidate groups
- 2. print legality failure reason
- 3. print layout propagation trace
- 4. print cost model table
- 5. compare profiler counters with expected bytes

Common Failure #1

- Pattern 은 맞지만 broadcast axis 가 다름
- Bias shape $[M, N]$ vs $[N]$ 차이
- 잘못 fuse 하면 semantic bug
- Verifier 에서 broadcast contract 를 검사

Common Failure #2

- Layout propagation 후 per-channel quant axis mismatch
- Scale tensor layout 업데이트 누락
- INT8/INT4 requant 에서 accuracy regression
- Axis mapping metadata 를 mandatory 로 뒀

Common Failure #3

- Fusion 으로 live range 가 증가해 SRAM spill 발생
- Traffic saving 보다 spill/repack 이 더 큼
- Cost model 없이 greedy fusion 금지
- Negative performance test 필요

실습 A

- 01_transformer_mlp_before.mlir 을 읽고 fusion candidate 를 표시
- root op, producer chain, consumer chain 작성
- multiple-use 여부 표시

실습 B

- Layout propagation graph 작성
- blocked layout 을 어디까지 유지할지 결정
- materialization point 와 reason attr 정의

실습 C

- 07_memory_traffic_calculator.py 실행
- unfused vs fused writes 비교
- SRAM budget 을 바꿔 fallback decision 작성

실습 D

- npu_graph.matmul_epilogue pseudo IR 작성
- epilogue/layout/accumulator/fusion_id attr 포함
- FileCheck positive/negative test 작성

Takeaways

- Fusion is a contract, not just a rewrite
- Layout propagation removes hidden copy/repack overhead
- Transformer MLP is the best practice case for NPU fusion analysis
- Cost model must include bytes, cycles, SRAM, layout, quantization

Final output: fused pseudo IR + layout propagation graph + cost table.

References

- MLIR Linalg Dialect
- MLIR Transform Dialect
- MLIR Data Layout Modeling
- MLIR MemRef Dialect
- MLIR Transform Tutorial Chapter 0